

TECHNICAL REPORT

Aluno: Cecília Maria Lima da Silva

1. Introdução

Neste projeto foi utilizado o dataset **Adult Income**, amplamente empregado em estudos de aprendizado de máquina, cujo objetivo é classificar indivíduos com base em sua renda anual, determinando se esta é superior ou inferior a 50 mil dólares.

O conjunto de dados apresenta características demográficas e socioeconômicas, tais como idade, nível de escolaridade, ocupação, estado civil, número de horas trabalhadas por semana, entre outras variáveis relevantes. A variável-alvo é categórica, sendo representada por duas classes: $\leq 50K$ e $> 50K$, caracterizando o problema como uma tarefa de **classificação binária**.

O objetivo principal deste trabalho consiste na construção de um modelo de classificação utilizando o algoritmo **K-Nearest Neighbors (KNN)**, avaliando seu desempenho a partir de diferentes técnicas de pré-processamento, análise de hiperparâmetros e métodos de validação.

Ao longo do desenvolvimento, foram realizadas etapas fundamentais como:

- análise exploratória dos dados;
- tratamento de valores ausentes;
- codificação de variáveis categóricas;
- aplicação de técnicas de normalização;
- treinamento e avaliação do modelo KNN;
- análise da influência do parâmetro K ;
- validação cruzada;
- otimização de hiperparâmetros com Grid Search.

Por fim, os resultados foram analisados criticamente com o intuito de compreender o comportamento do modelo e identificar possíveis limitações e melhorias.

2. Observações

Durante a análise inicial do dataset, foi identificado que algumas variáveis apresentavam valores ausentes representados pelo caractere "?". Para garantir a

qualidade dos dados, esses valores foram convertidos para NaN e posteriormente removidos com o uso da função `dropna()`.

Além disso, observou-se a presença de diversas variáveis categóricas, tais como:

- `workclass`
- `education`
- `occupation`
- `marital-status`
- `native-country`

Como o algoritmo KNN opera exclusivamente com dados numéricos, foi necessário aplicar a técnica de **One-Hot Encoding**, transformando essas variáveis em representações numéricas.

Outro ponto relevante refere-se à sensibilidade do KNN à escala dos dados, uma vez que o algoritmo utiliza métricas de distância para realizar classificações. Dessa forma, foram aplicadas três técnicas distintas de normalização:

- transformação logarítmica (Log);
- normalização Min-Max;
- padronização Z-score.

Não foram observados erros críticos durante a execução do projeto, sendo necessárias apenas correções relacionadas à visualização dos gráficos.

3. Resultados e discussão

3.1 Carregamento e Exploração Inicial do Dataset

A etapa inicial do projeto consistiu na leitura e inspeção do dataset utilizando a biblioteca `pandas`. Foram empregadas funções como `head()`, `info()` e `value_counts()` com o objetivo de compreender a estrutura dos dados, identificar tipos de variáveis e verificar a distribuição da variável-alvo.

Observou-se que o conjunto de dados é composto por uma combinação de atributos numéricos e categóricos, sendo estes últimos predominantes. A variável-alvo, denominada `income`, apresenta duas classes distintas (`<=50K` e `>50K`), caracterizando o problema como uma tarefa de classificação binária.

Além disso, foi possível perceber que a distribuição das classes não é perfeitamente equilibrada, o que pode influenciar o desempenho do modelo, especialmente na identificação da classe minoritária.

3.2 Tratamento de Valores Ausentes e Pré-processamento

Durante a análise exploratória, identificou-se a presença de valores ausentes representados pelo caractere "?". Esses valores foram convertidos para NaN e posteriormente removidos utilizando a função `dropna()`. Essa etapa foi fundamental para garantir a consistência do dataset, uma vez que algoritmos de aprendizado de máquina não lidam diretamente com valores ausentes.

Em seguida, as variáveis categóricas foram transformadas em formato numérico por meio da técnica de **One-Hot Encoding**, utilizando a função `get_dummies()`. Essa transformação é necessária para permitir que o algoritmo KNN, que opera em espaço numérico, possa calcular distâncias entre as amostras.

Embora essa técnica aumente significativamente a dimensionalidade do dataset, ela preserva a informação sem introduzir ordem artificial entre categorias, o que é adequado para esse tipo de problema.

3.3 Análise de Correlação e Relevância dos Atributos

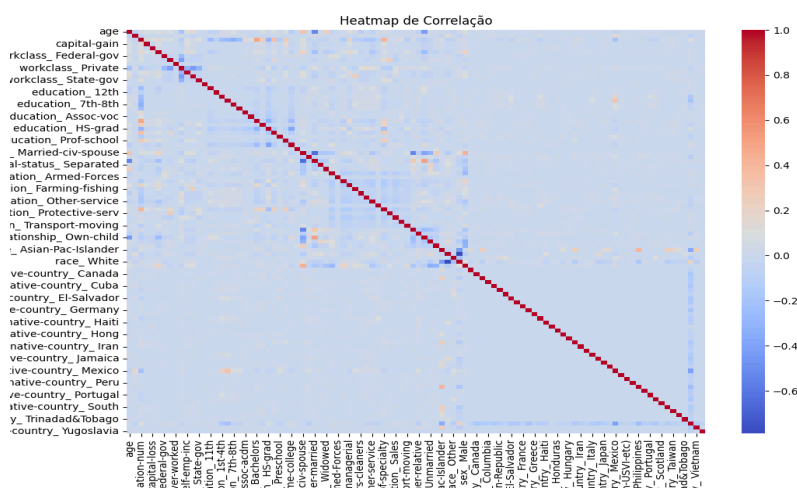


Figura 1 – Heatmap de correlação entre os atributos do dataset.



A análise da matriz de correlação permitiu avaliar a relação linear entre as variáveis do dataset. Observou-se que a maioria dos atributos apresenta baixa correlação linear entre si, o que indica baixa redundância entre as variáveis. Esse comportamento é esperado, especialmente após a aplicação do One-Hot Encoding, que gera variáveis binárias independentes.

Apesar disso, algumas variáveis numéricas, como idade (age), horas trabalhadas (hours-per-week) e ganhos de capital (capital-gain), demonstram correlações discretas, sugerindo que podem contribuir de forma complementar para o processo de classificação.

A baixa correlação global reforça a importância da utilização de múltiplos atributos pelo modelo KNN, uma vez que a separação das classes não depende de uma única variável dominante.

3.4 Separação entre Conjunto de Treino e Teste

Após o pré-processamento, os dados foram divididos em conjuntos de treino (75%) e teste (25%) utilizando a função `train_test_split()`.

Essa divisão é essencial para avaliar a capacidade de generalização do modelo, evitando que ele seja avaliado com os mesmos dados utilizados no treinamento.

O conjunto de treino foi utilizado para ajustar o modelo, enquanto o conjunto de teste foi empregado para avaliar seu desempenho em dados não vistos anteriormente.

3.5 Aplicação das Técnicas de Normalização

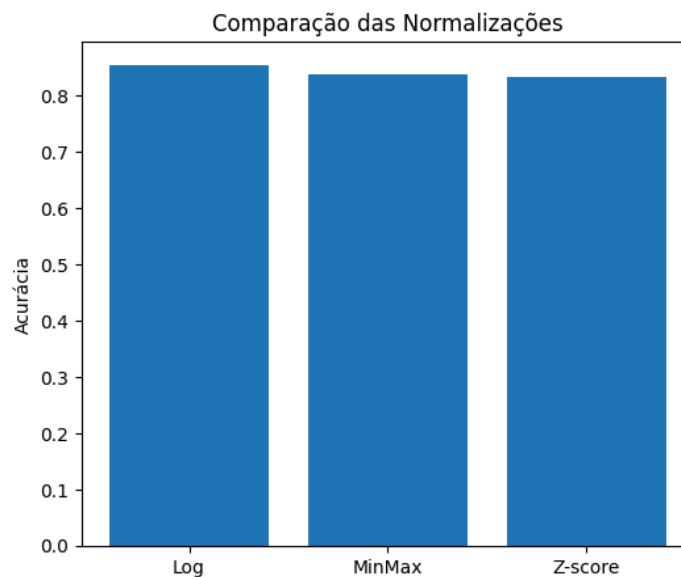


Figura 2 – Comparação das técnicas de normalização.

Considerando que o algoritmo KNN é baseado em medidas de distância, a escala dos atributos exerce influência direta no desempenho do modelo. Variáveis com maior magnitude podem dominar o cálculo das distâncias, comprometendo a qualidade da classificação.

Para minimizar esse problema, foram avaliadas três técnicas de normalização:

- **Transformação Logarítmica (Log)**
- **Min-Max Scaling**
- **Z-score (Padronização)**

Os resultados obtidos indicaram que:

- Log apresentou acurácia aproximada de **0.85**
- Min-Max apresentou aproximadamente **0.84**
- Z-score apresentou aproximadamente **0.83**

A superioridade da transformação logarítmica pode ser explicada pela sua capacidade de reduzir a assimetria dos dados e suavizar a influência de valores extremos, tornando a distribuição mais homogênea e adequada para o cálculo de distâncias.

3.6 Implementação do Modelo KNN

O algoritmo *K-Nearest Neighbors* foi implementado utilizando a biblioteca *scikit-learn*. Esse algoritmo classifica uma nova amostra com base na proximidade (distância) em relação aos seus vizinhos mais próximos.

O modelo foi treinado utilizando diferentes valores do hiperparâmetro **K (número de vizinhos)**, variando de 1 a 30, com o objetivo de analisar seu impacto no desempenho.

3.7 Análise da Influência do Parâmetro K

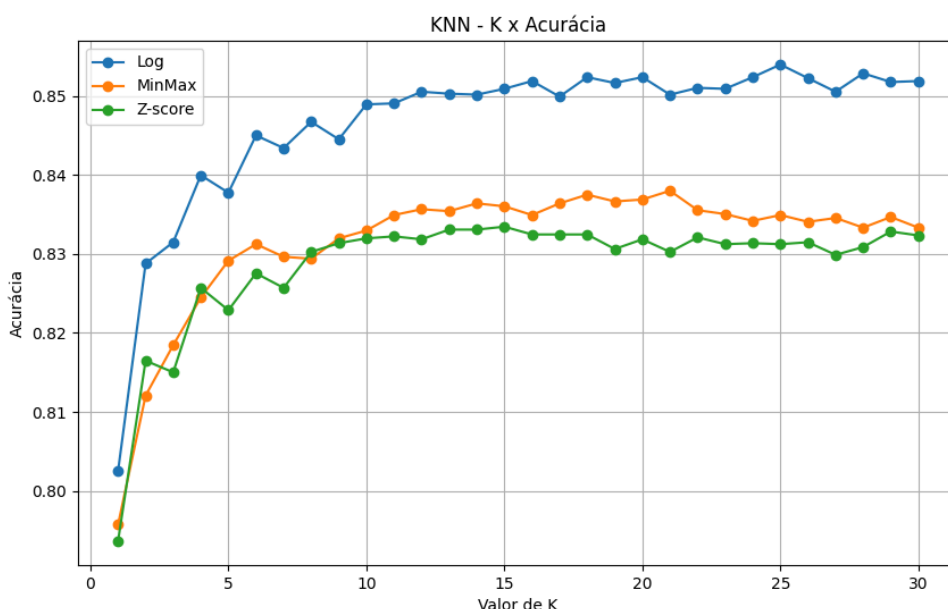


Figura 3 – Relação entre o valor de K e a acurácia do modelo.

A análise do gráfico revelou um comportamento típico do algoritmo KNN:

- Para valores muito baixos de K (ex: $K=1$), o modelo apresenta alta variabilidade, sendo altamente sensível a ruídos (**overfitting**).
- À medida que K aumenta, o modelo se torna mais estável, reduzindo a influência de pontos isolados.
- Para valores muito altos de K, o modelo tende a suavizar excessivamente as fronteiras de decisão (**underfitting**).

Os melhores resultados foram observados para valores intermediários de K (entre aproximadamente 20 e 25), onde o modelo alcançou acurácia próxima de **85%**.

3.8 Identificação da Melhor Parametrização

A partir da análise visual do gráfico, foi possível identificar o intervalo de valores de K que proporciona melhor desempenho.

A escolha de um valor intermediário representa um equilíbrio entre:

- *sensibilidade local (K pequeno)*
- *generalização global (K grande)*

Essa análise é essencial, pois o valor de K influencia diretamente a capacidade de classificação do modelo.

3.9 Validação Cruzada

A validação cruzada foi aplicada com o objetivo de avaliar o modelo de forma mais robusta.

Esse método divide o dataset em múltiplos subconjuntos (folds), garantindo que todas as amostras sejam utilizadas tanto para treinamento quanto para validação.

Os resultados obtidos indicaram baixa variabilidade entre os folds, evidenciando que o modelo apresenta boa estabilidade e capacidade de generalização.

3.10 Otimização com Grid Search

O Grid Search foi utilizado para encontrar automaticamente o melhor valor de K . Os resultados obtidos confirmaram a análise visual realizada anteriormente, reforçando a confiabilidade do processo de seleção de hiperparâmetros. Essa etapa automatizada garante uma escolha mais precisa e reduz a subjetividade da análise manual.

3.11 Comparação das Técnicas de Normalização

A comparação entre as três técnicas mostrou que todas apresentaram bom desempenho, com diferenças relativamente pequenas entre elas.

Entretanto, a transformação logarítmica se destacou como a melhor abordagem, indicando que o tratamento da distribuição dos dados é um fator relevante para o desempenho do KNN.

3.12 Matriz de Confusão e Análise de Desempenho

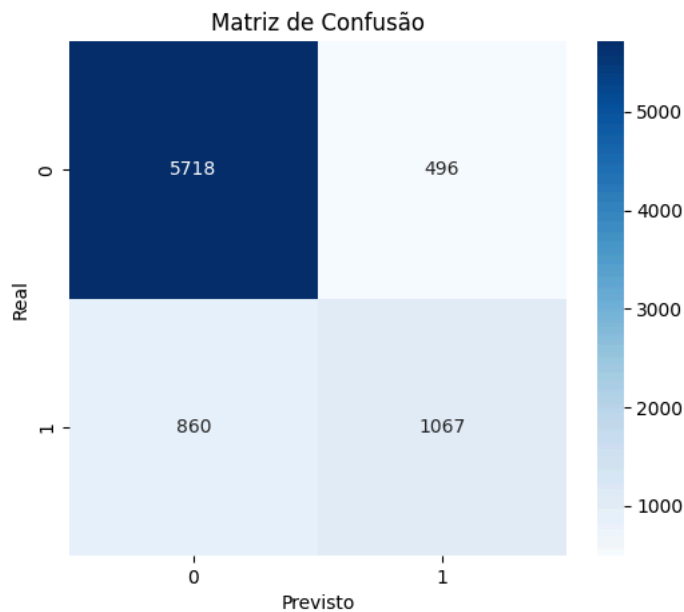


Figura 4 – Matriz de confusão do modelo KNN.

A matriz de confusão apresentou os seguintes resultados:

- Verdadeiros negativos: **5718**
- Falsos positivos: **496**
- Falsos negativos: **860**
- Verdadeiros positivos: **1067**

A análise mostra que o modelo apresenta melhor desempenho na classe majoritária ($\leq 50K$), com maior taxa de acerto.

Por outro lado, observa-se maior dificuldade na identificação da classe minoritária ($> 50K$), evidenciada pelo número de falsos negativos.

Esse comportamento pode ser explicado pelo **desbalanceamento das classes**, onde a classe majoritária influencia mais o processo de aprendizagem do modelo.

4. Conclusões

O modelo K-Nearest Neighbors (KNN) demonstrou desempenho satisfatório na classificação do dataset Adult Income, alcançando aproximadamente 85% de acurácia,

o que evidencia sua eficácia em problemas de classificação binária com múltiplos atributos heterogêneos.

A análise do hiperparâmetro K revelou que valores intermediários proporcionam melhor equilíbrio entre variância e viés, resultando em maior capacidade de generalização. Além disso, verificou-se que o pré-processamento dos dados exerce papel determinante no desempenho do modelo, com destaque para a transformação logarítmica, que apresentou superioridade ao reduzir a influência de outliers e melhorar a distribuição dos atributos.

A avaliação por meio da matriz de confusão indicou uma tendência do modelo em favorecer a classe majoritária, evidenciando limitações na identificação da classe minoritária. Esse comportamento sugere impacto direto do desbalanceamento dos dados, apontando para a necessidade de estratégias adicionais para melhoria do desempenho.

De maneira geral, conclui-se que o algoritmo KNN, apesar de sua simplicidade, apresenta resultados consistentes e confiáveis quando associado a um adequado tratamento dos dados e escolha criteriosa de hiperparâmetros, consolidando-se como uma abordagem válida para o problema proposto.

5. Próximos passos

Como continuidade do projeto, recomenda-se a aplicação de algoritmos mais robustos, como Random Forest, Support Vector Machines (SVM) e Redes Neurais, a fim de comparar seus desempenhos com o KNN e verificar possíveis ganhos de acurácia.

Além disso, torna-se relevante a utilização de técnicas de balanceamento de classes, como oversampling ou undersampling, com o objetivo de melhorar a identificação da classe minoritária, reduzindo o número de falsos negativos.

Outro ponto importante consiste na realização de seleção de atributos, visando reduzir a dimensionalidade do dataset e eliminar variáveis redundantes, o que pode impactar positivamente tanto no desempenho quanto na eficiência computacional do modelo.

Por fim, sugere-se o aprofundamento na otimização de hiperparâmetros e na análise de diferentes métricas de distância, buscando aprimorar ainda mais a capacidade preditiva do algoritmo.